

G.A.T.E.S. — Gate 1

What it checks, what it measured, and what changed when it was switched on

Generated 2026-08-15 · gates @ main · 249 gate tests, 63 host integration tests · every number below is a recorded measurement or a cited published figure

1. The defect, in one line of the host scaffold

```
def execute_code(code_str, timeout=60, MAX_LEN=1000):
    ...
    except Exception as e:
        output_capture.write(f"[CODE EXECUTION ERROR]: {str(e)}")
    # appended AFTER the prints
    return output_capture.getvalue()[:MAX_LEN] # ...then sliced off
```

The writing agent's entire experimental record was 1,000 characters, and the crash marker was appended after the program's own output — so on any run that printed more than that, the marker fell off the same slice and the solver's only crash test never fired.

measured in the archived run	value
Reward score on a run raising <code>NameError</code> every attempt	0.95 → 0.98 → 1.0
Test accuracy reported in the paper	81.60% (never measured)
Speedup reported	13.61× (never measured)
Ablation collapse reported	39.20% (never measured)
Archived runs usable of seven	1

2. The run

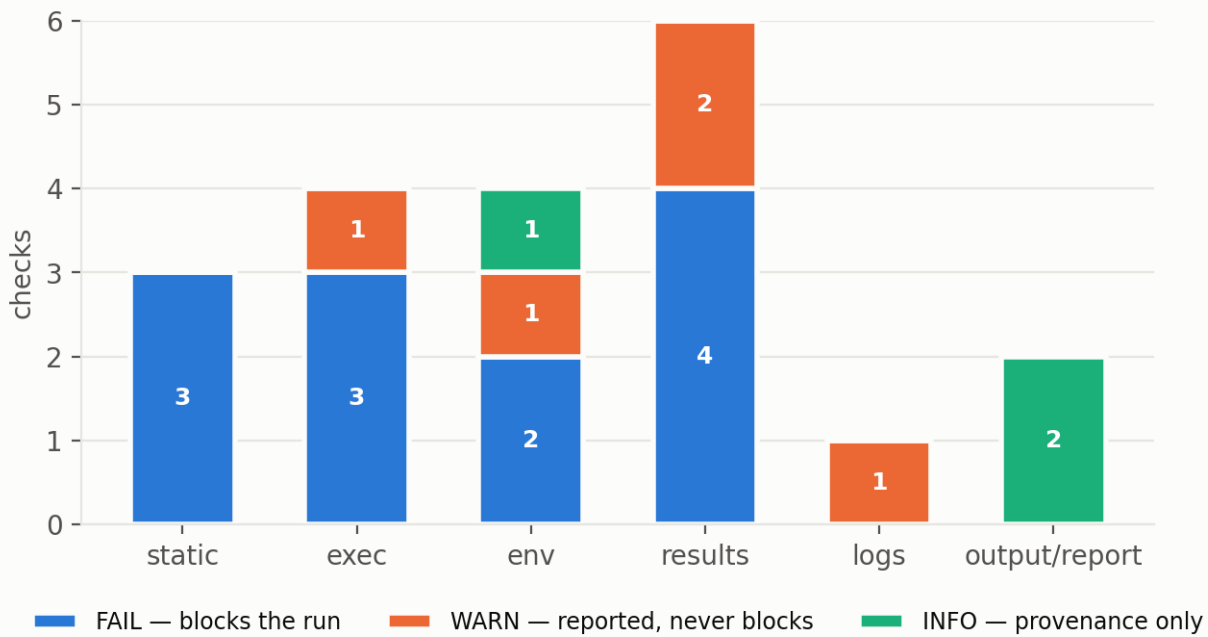
No ablation run record found. Sections 2-5 are generated from one.

5. The full check inventory

Twenty deterministic checks in six families, plus one model-assisted check. The run fails if and only if a **FAIL** check fails; **WARN** and **INFO** are reported and can never change a verdict.

Gate 1 check inventory — 20 checks in 6 families

Severity decides the verdict: FAIL blocks, WARN and INFO never can



check	family	severity	fails when
static.syntax_valid	static	FAIL	Source does not compile. Before execution — a broken program costs no compute.
static.no_unbound_names	static	FAIL	A name is read but bound nowhere, resolved with symtable. The archived run's exact failure: hidden_dim read in forward(), assigned in no enclosing scope.
static.no_banned_calls	static	FAIL	exit() / sys.exit() would forge a clean exit code. Removes the cheapest way to fake success.
exec.exit_code_zero	exec	FAIL	The process exited non-zero. Reads the exit code, not a substring of stdout.
exec.no_uncaught_exception	exec	FAIL	An exception escaped the run. Recorded by the harness in the child process.
exec.completed_within_budget	exec	FAIL	The process group was killed on timeout. Kills the group, so a runaway cannot survive its own timeout.
exec.no_swallowed_traceback	exec	WARN	A traceback appears in either stream on a run that still exited 0. Both streams: except Exception as e: print(e) puts the evidence on stdout.
env.clean_namespace	env	FAIL	The initial globals contained anything beyond harness-provided names. Makes “the variables are real” a runtime property, not a hope.
env.code_identity	env	FAIL	The source the child hashed differs from the source the parent wrote. Makes “hashed to the run that produced it” checkable rather than assumed.

<code>env.seed_recorded</code>	env	WARN	No seed was declared. A run with no seed cannot be re-executed to check its own numbers.
<code>env.provenance</code>	env	INFO	Python version, platform, device, library versions, code SHA-256.
<code>results.contract_present</code>	results	FAIL	<code>results.json</code> is absent or unparseable. Nothing was recorded, so nothing is citable.
<code>results.expected_keys_present</code>	results	FAIL	A key the plan declared is missing.
<code>results.values_computed</code>	results	FAIL	A recorded value is a source literal at its call site. The strongest claim in Gate 1. <code>record_result("k", acc)</code> passes; <code>record_result("k", 0.816)</code> fails, as does <code>round(0.8160, 3)</code> — constant folding does not launder a typed number.
<code>results.values_finite</code>	results	FAIL	A value is NaN or infinite.
<code>results.single_observation</code>	results	WARN	A key was recorded repeatedly with a changing value. Every <code>record_result</code> call is retained, so a best-epoch number reported as a final-epoch one is visible.
<code>results.non_degenerate</code>	results	WARN	Exact zero, exact one, or chance level when the class count is known. Answers the limitation AutoResearchClaw reports for value registries: a registry passes real zeros.
<code>logs.no_error_signals</code>	logs	WARN	Numerical warnings, device failures, non-convergence, printed exceptions, ERROR-level logging. Tuned for precision: <code>ValueError: is flagged</code> , <code>Mean Squared Error: is</code> <code>not</code> .
<code>output.untruncated</code>	output	INFO	<code>stdout/stderr</code> byte counts; asserts no truncation was applied. Retires the 1,000-character ceiling.
<code>report.fixes_grounded</code>	report	INFO	Generated fixes cited something the run does not contain. Records that the model's text was discarded and why.
<code>logs.model_error_signals</code>	logs (model)	WARN	Lines the pattern set could not reach, read by the LLM layer. WARN by construction — <code>model_warning()</code> takes no severity parameter, so no model output can reach the verdict.

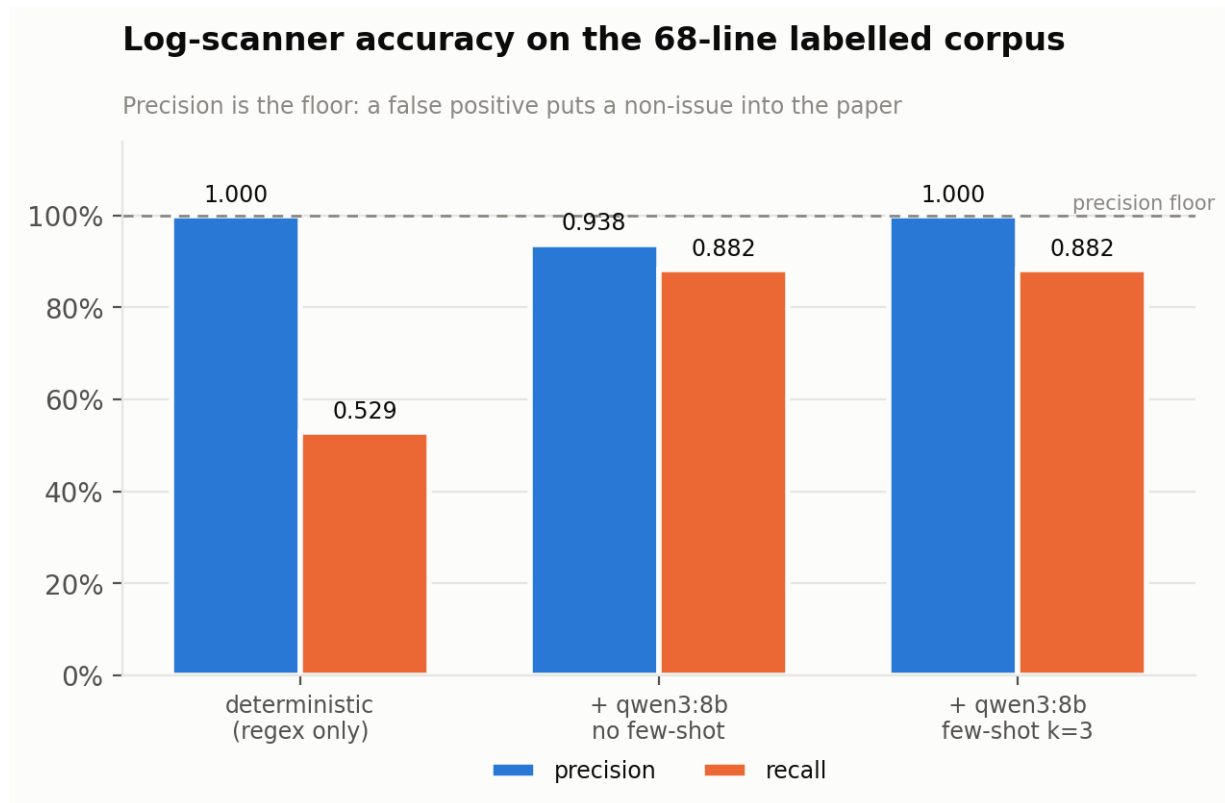
Why the severity split is load-bearing. The LLM layer is a required part of Gate 1 — the feedback report is the loop's return path to the ML engineer, and no template writes “bind `hidden_dim` before use, or pass it into `GCN.__init__`”. It coexists with a model-free verdict because of where it sits and what severity it can emit: model findings are WARN by construction, and report generation runs after `decide()` has already fixed the verdict. A test parses the module's AST to assert `Severity`. FAIL never appears in an expression there.

6. Metrics the gate records per attempt

metric	definition
trace_id	sha256(run_id, key, lineno) — binds one value to one execution. Two runs of identical source give different trace ids, which is what makes a backfilled number detectable.
chain_integrity	fraction of values whose provenance chain (task → command → log → value) is complete, with the missing link named per key.
citable	false on a rejected run's registry, so a consumer that ignores the verdict still cannot cite it.
arg_kind	computed or literal, from static analysis of the record_result call site.
call_count	times a key was recorded, with the value span — a best-epoch number reported as final is visible.
code_sha256	hashed by the child that ran it and compared to what the parent wrote.
model.calls / degraded	what the LLM layer spent, and whether any call failed, so a thinner report is never mistaken for a complete one.

7. The log scanner, measured

68 labelled lines, 34 of them error signals, 16 of those outside anything a regex was going to reach. Sixteen lines come verbatim from the archived run.



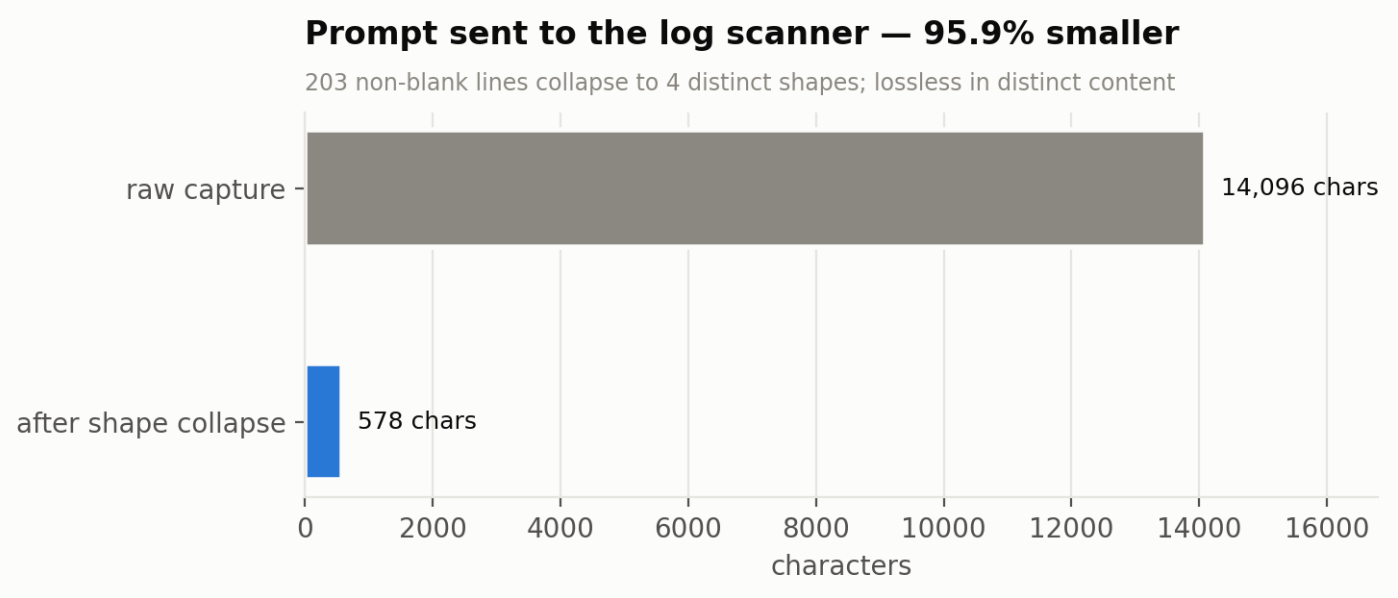
scanner	precision	95% CI	recall	95% CI
deterministic (regex only)	1.000	0.824–1.000	0.529	0.367–0.685

+ qwen3:8b, no few-shot	0.938	0.798–0.983	0.882	0.734–0.953
+ qwen3:8b, few-shot k=3	1.000	0.886–1.000	0.882	0.734–0.953

The model tier raised recall from 0.529 to 0.882 at no cost in precision (1.000 → 1.000), taking F1 from 0.692 to 0.938. It runs on a local 8B model at zero marginal cost, so this is applied to every attempt rather than sampled.

Retrieval is what bought the precision back. Without few-shot examples the same model scored 0.938 precision and flagged arch-009, arch-014 — framework noise, one of them the cuDNN registration notice the corpus was built around. Balanced retrieval (k signals and k negatives, never whichever scores highest) removed both while recall held at 0.882. The nearest *negative* was the useful neighbour, which is the whole reason the exemplar bank is half noise.

Still missed at k=3: arch-004, dev-004, dev-005, gap-007 — 4 of 34 signals.

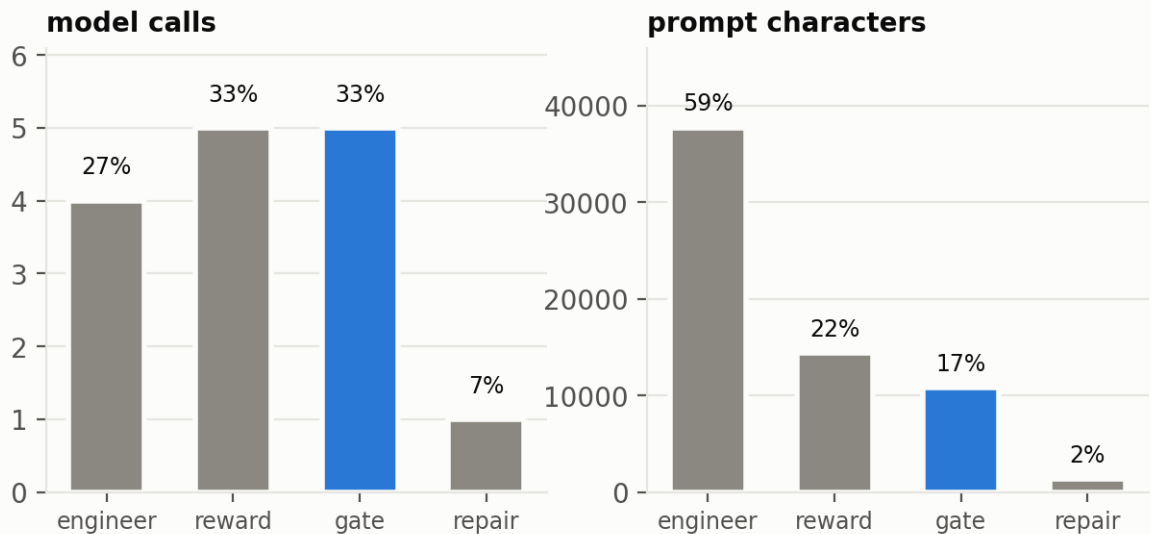


Before the model reads anything, the log is collapsed to its distinct shapes: **203 non-blank lines to four**, a 95.9% smaller prompt. The collapse is lossless in distinct content — every shape survives with its first real line number, so nothing a scanner could have flagged disappears.

8. What the layer costs

Gate 1 is a third of the calls and a sixth of the tokens

Measured over five executions of one solver phase; blue is the gate



Measured over five executions of one solver phase: Gate 1's own model calls are **a third of the calls but a sixth of the tokens**. That is why the gate can run on qwen3:8b on an 8 GB laptop GPU — 5.5 GB resident, ~16s a call, zero marginal cost — while the engineer stays on a stronger model. Putting the *engineer* on a weak model is the tempting economy and the wrong one: it fails the gate more often, and every rejection costs a fixed call, a shadow-reward call and another turn.

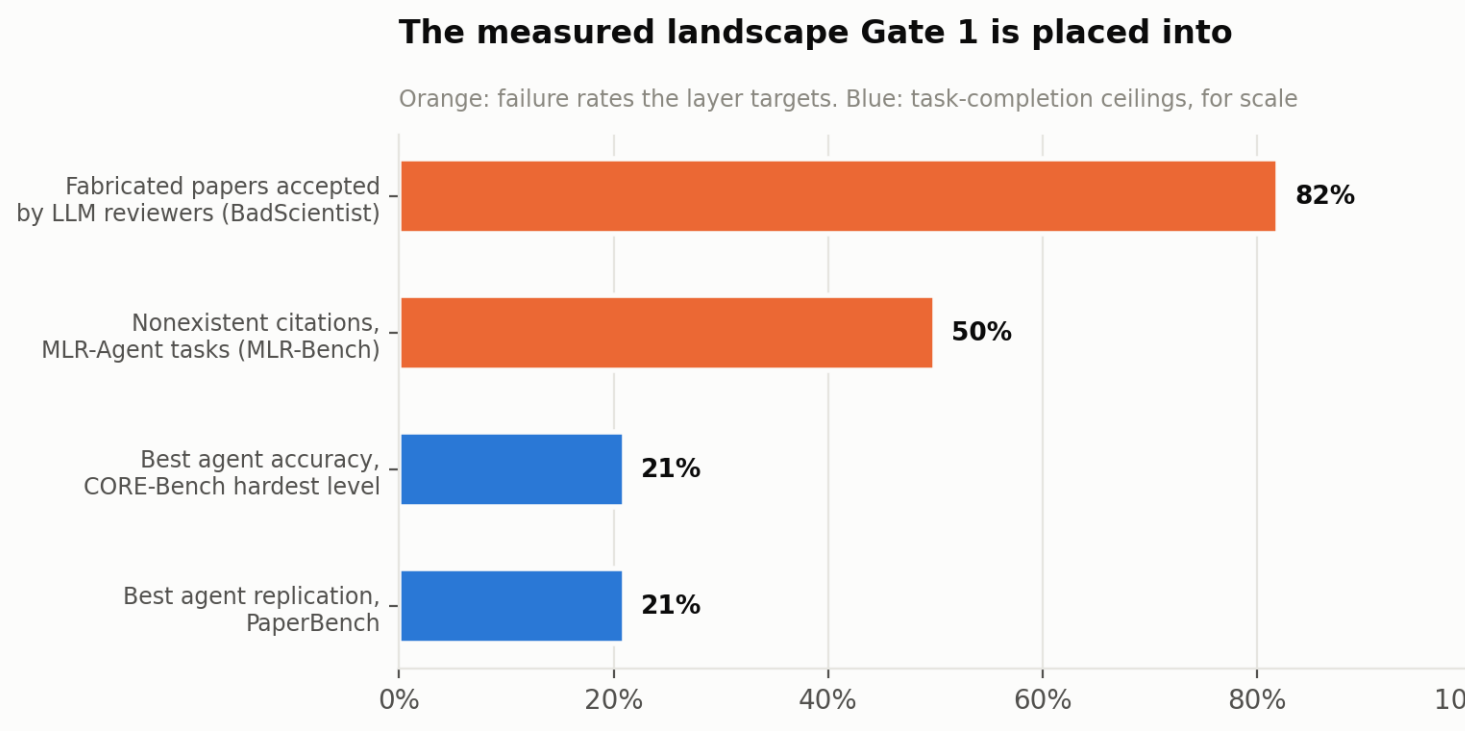
9. Defects the runs found

Nine, fixed. Four appeared only against a real model, and two of those only against the weaker one.

defect	found by	consequence
Unparseable reply executed an empty experiment	scripted e2e	Gate reported "you never called record_result()" when the command had not parsed
Evidence bundle showed warning counts without evidence	scripted e2e	The writer was told to disclose warnings it could not see
Fallback note reported an outage that had not happened	scripted e2e	Three different situations under one message
Prose in backticks parsed as code	gemini-3.5-flash	A good fix discarded over ['so', 'that', 'it', 'when', 'executing']
Prompt leaked the gate's artifact paths	gemini-3.5-flash	Told the engineer to edit /home/.../attempt_03/experiment.py
Scan prompt offered two numbers per row	qwen3:8b	The weaker model reported the file line, not the row index; a correct finding was discarded

Output cap unreachable for most backends	qwen3:8b	One call generated 21,858 tokens for an answer whose correct form is []
Rate-limit retry killed the phase in 25s	gemini-3.5-flash	Flat 5s × 5 against a per-minute meter
Five config keys reached nothing	integration audit	A config declaring a lit-review backend did not get one

10. Where this sits against the published benchmarks



source	what it measures	reported
MLR-Bench arXiv 2505.19955	Four fact-based hallucination types in AI-generated papers.	Faked results and hallucinated methodology each in more than half of 10 tasks; almost all AI Scientist V2 papers contain both . Nonexistent citations in 50% of MLR-Agent tasks.
BadScientist arXiv 2510.18003	Whether fabricated papers pass multi-model LLM review.	Accepted at up to 82.0% ; detection “barely exceeding random chance”.
CORE-Bench arXiv 2409.11363	Computational reproducibility by agents.	Best agent 21% on the hardest level; correctness judged against a 95% prediction interval over three manual reproductions.
PaperBench arXiv 2504.01848	Replicating AI research end to end.	Best agent 21.0% ; ML PhDs 41.4% best-of-3 after 48 hours.

BadScientist is the reason Gate 1's verdict consults no model: if LLM reviewers detect fabrication at barely above

chance, a validity layer built on model judgement inherits that ceiling.

Naming. This project's design documents paraphrase MLR-Bench's taxonomy. Its published names are *faked experimental results, hallucinated methodology, incorrect citations, mathematical errors*. "Silent failure scored as success" is a cause of the first in their scheme, not a fifth class, and any "eliminates N of four" claim should say so.

11. What Gate 1 does not claim

- It does not check whether a result is *plausible* — that is Gate 2.
- It does not read the manuscript — that is Gate 3.
- The static tier is conservative on purpose; the runtime tier catches what it declines to claim, one execution later.
- The scanner's recall is bounded by the lines it was shown, and that count is recorded so the claim cannot exceed the evidence.
- A mean computed inside the experiment over a silently shortened list arrives as one value the gate cannot see behind.
- The ablation is **n = 1 at temperature 0**. It shows what happened on one task with one model; it is not a rate.
- n is still 1 for the archived-run comparison — the re-run has not been done.

12. Reproducing this

```
./tools_local_model.sh start          # local qwen3:8b, no API key
python tools_ablation.py --model qwen3-8b-local --turns 4
python -m rig.corpus                  # deterministic scanner baseline
python -m rig.gate1_loop              # the five loop scenarios
python reports/make_charts.py && python reports/make_report.py
python reports/make_deck.py
pytest                               # 249 gate tests
```

Sources: MLR-Bench (2505.19955), BadScientist (2510.18003), CORE-Bench (2409.11363), PaperBench (2504.01848), RE-Bench (2411.15114), AutoResearchClaw (2605.20025), ScientistOne (2605.26340), SAGE (2606.31478).